MATCHREAD

MatchStat API Architecture Corrections & Implementation Guide

Engineer handoff - bracket, draw synchronization, live mapping, websocket recovery, and coverage fallback

Purpose
Convert MatchStat's answers into an implementation plan for MatchRead. This document separates provider-confirmed behavior from MatchRead engineering recommendations and turns each discovery into concrete coding work, endpoint flows, validation checks, and launch gates.

Prepared for MatchRead launch engineering
August 2026

Executive Summary

Overall assessment: A- provider fit for MatchRead, with several architecture corrections required before relying on the integration for a Grand Slam launch.

Core architecture decision
Treat MatchStat as three cooperating systems, not one fixtures feed: (1) Core API for tournament/player/fixture/result metadata, (2) Draw API for official bracket structure and positions, and (3) Extend Live API + Socket.IO for real-time scoring and timeline updates.

The nine discoveries and their engineering consequence

#	Discovery	Required correction	Launch priority
1	Bracket source	Use /draws, not fixture-order inference	P0
2	Draw position exists	Build deterministic bracket slots from draw values	P0
3	Official seeds	Persist seed1/seed2 separately from entry type	P0
4	Lucky losers/replacements	Detect draw player changes during refresh	P0
5	Socket IDs differ	Create fixture-to-live EventMapper service	P0
6	No tournament socket room	Join individual live events for active MatchRead matches	P1
7	No replay	REST re-sync immediately after reconnect	P0
8	Coverage not guaranteed	Gracefully fall back to REST polling	P1
9	Grand Slam timing not guaranteed	Poll draw publication/change state; never hardcode timing	P0

Target data flow

```
Tournament
  |
  v
/draws  ---> canonical bracket structure
```



Definition of done before US Open reliance

- Bracket renders correctly from /draws without any fixture-order assumptions.
- Replacement detection updates a player while preserving the same bracket slot and downstream bracket relationships.
- Every active live-capable match can be mapped from the core fixture representation to the Extend Live event ID.
- A websocket disconnect/reconnect test proves that REST re-sync repairs missed state before socket updates resume.
- No-live-event cases degrade to polling without breaking the match page or bracket.
- Draw publication/change polling can detect the official draw becoming available without a fixed provider delay assumption.

Discovery 1: Do not build the bracket from fixtures

Impact: Critical architecture **Priority:** P0 - immediate

Provider-confirmed
MatchStat explicitly states that fixtures are not a bracket. The draw endpoint must be the canonical source for bracket structure.

Correction required

Remove any code path where first-round fixture order, fixture creation order, match date, or fixture ID is used to infer bracket topology. Fixtures may still enrich matches, but they must not define bracket position.

Endpoint flow

```
GET /draws
  -> normalize draw records
  -> derive bracket slot / round / participants
  -> persist canonical draw snapshot
  -> attach core fixture metadata when available
```

Step-by-step coding work

1. Locate the current bracket ingestion path and identify every place fixtures are sorted or paired to create bracket rows.
2. Create a DrawProvider client method whose only responsibility is fetching /draws for the selected tournament/season.
3. Create a normalizeDrawRecord() boundary that converts provider fields into MatchRead-owned types. Do not let raw provider payload shape leak into React components or scoring logic.
4. Persist a draw snapshot keyed by tournament + draw/competition identity + fetchedAt/providerUpdatedAt where available.
5. Generate bracket nodes from the normalized draw records. Fixture data becomes optional enrichment attached after topology exists.
6. Gate publication: if the draw endpoint has not returned a usable main draw, show "Draw not yet published" rather than fabricating a bracket from fixtures.
7. Delete or disable bespoke fixture-pairing heuristics once the draw path passes parity tests.

Suggested internal types

```
type DrawNode = {
  tournamentId: string;
  providerDrawPosition: number | string;
  round: string;
  player1Id?: string;
  player2Id?: string;
  seed1?: number | null;
  seed2?: number | null;
  entryType1?: 'WC'|'Q'|'LL'|'PR'|null;
  entryType2?: 'WC'|'Q'|'LL'|'PR'|null;
```

```
  coreFixtureId?: string | null;
};
```

Tests

- Fixture response shuffled randomly: bracket output must remain identical.
- Fixture IDs changed: bracket slot assignments must remain identical.
- Fixtures unavailable but /draws available: bracket must still render.
- /draws unavailable: bracket must remain unpublished, not inferred.

Discovery 2: Draw position exists

Impact: High - removes ordering ambiguity **Priority:** P0 - immediate

Provider-confirmed
MatchStat reports a draw field that provides official draw position/order information.

This is the critical field that should replace hand-authored ordering logic. The engineer should still verify its exact semantics for the target draw size before encoding formulas globally.

Correction required

- Treat draw as provider-owned bracket ordering metadata.
- Never convert draw to an assumed seat formula until a live sample proves whether the value is per-match, per-player, round-relative, or global.
- Keep the raw provider draw value in storage so future normalization changes are reversible.

Implementation probe before production

```
GET /draws  # on a known 32/64/96/128-player event

Check:
1. number of distinct draw values in opening round
2. whether draw increments by match or by player/seat
3. whether later rounds retain related draw values
4. how byes are represented
5. whether women/men or singles/doubles use the same semantics
```

Recommended normalizer

```
raw /draws record
   |
   v
DrawNormalizer
   |-- preserve providerDrawPosition
   |-- assign canonical round
   |-- assign MatchRead slot index
   |-- validate uniqueness
   |-- emit warnings for collisions/gaps
   v
BracketNode[]
```

Validation gates

- No duplicate canonical slot in the same round unless the provider semantics explicitly require it.
- Expected opening-round node count matches draw size after accounting for byes.
- Each later-round node has a deterministic parent relationship.
- Provider raw draw value is logged on any normalization error.

Discovery 3: Official seeds are available

Impact: High product correctness **Priority:** P0

Provider-confirmed
seed1 and seed2 are official tournament seeds. WC, PR, Q, and LL are entry/status designations, not seed numbers.

Correction required

Separate seed and entry type in the MatchRead schema and UI. Do not overload one field to show both.

Step-by-step coding work

8. Add or confirm nullable numeric seed fields for each bracket participant.
9. Add a separate entryType enum/string for WC, PR, Q, LL and any other provider designation observed.
10. At ingestion, parse seed values as numbers where appropriate; never store "WC" or "Q" in the seed column.
11. At display time, show a seed number as the ranking badge and optionally an entry-type badge beside the player name.
12. At replacement time, refresh both seed and entry type from the newest draw snapshot rather than copying the outgoing player metadata.

Schema concept

```
participant_seed SMALLINT NULL
participant_entry_type TEXT NULL  -- WC | PR | Q | LL | ...
provider_player_id TEXT
provider_draw_position TEXT/INT
```

Tests

- Seeded wild card if provider ever returns one: seed and entry type can coexist.
- Qualifier without seed: displays Q, no fake numeric seed.
- Replacement: incoming LL shows LL and correct seed state.

Discovery 4: Lucky losers and replacements are player changes, not events

Impact: Critical synchronization **Priority:** P0

Provider-confirmed
There is no dedicated LuckyLoserEvent to wait for. MatchRead must detect changes in draw participants across refreshes.

Correction required

The draw refresh worker must perform snapshot diffing. A bracket slot is the stable identity; the player occupying that slot can change before the match begins.

Required flow

```
scheduled /draws refresh
   |
   v
normalize current draw
   |
   v
compare against previous snapshot by stable slot
   |
   +-- same player -> no-op
   |
   +-- different player -> replacement detected
          |
          +-- update bracket participant
          +-- refresh seed / entry type
          +-- invalidate fixture/live mapping if players changed
          +-- preserve user bracket semantics per lock policy
          +-- audit/log replacement
```

Step-by-step coding work

13. Create DrawSnapshot and DrawSlotSnapshot records or an equivalent immutable history mechanism.
14. Compare snapshots using stable tournament + round + canonical slot identity, not player ID.
15. When participant identity changes, write a replacement audit record containing old player, new player, slot, detection timestamp, and provider payload reference/hash.
16. Invalidate any EventMapper result tied to the old player pair because the live event lookup may now resolve differently.
17. Re-run fixture association if the core fixture also changed.
18. Decide bracket-pick behavior explicitly: before lock, update the selectable player automatically; after lock, preserve the user's submitted pick identity according to the MatchRead scoring rule rather than silently rewriting a prediction.
19. Expose replacement state to the admin/founder dashboard so pre-event anomalies are visible.

Failure modes to prevent

- Outgoing player remains visually cached after provider replacement.
- Live mapping still points to a match containing the withdrawn player.
- Submitted user pick is silently changed to the replacement without an explicit product rule.
- Replacement creates a second bracket slot instead of updating the existing slot.

Discovery 5: Core fixture ID and Socket event ID are different

Impact: Critical live integration **Priority:** P0

Provider-confirmed
Core fixture identifiers and Extend Live/Socket event identifiers differ, and MatchStat reports no direct ID-mapping endpoint. Their recommended resolution uses player1 + player2 + date against event/get.

Required service: EventMapper

Make this a first-class backend service with its own cache, matching policy, observability, and retry logic. Do not embed lookup logic in a page component or socket handler.

Endpoint flow

```
Core fixture
  -> player1
  -> player2
  -> scheduled date/time
       |
       v
    event/get
       |
       v
candidate Extend Live event(s)
       |
       v
deterministic match validation
       |
       v
persist socketEventId
       |
       v
join-event(socketEventId)
```

Step-by-step coding work

20. Create EventMapper.resolve(coreFixture) on the server/backend only.
21. Normalize player IDs/names according to the provider fields available in both systems. Prefer provider IDs when compatible; otherwise use conservative matching and additional checks.
22. Query event/get using the documented player/date flow.
23. Validate candidate tournament/date/player pairing before accepting the event ID. Never accept the first fuzzy match blindly.

24. Persist mapping: coreFixtureId -> socketEventId, mappedAt, confidence/method, and source attributes.
25. Cache positive mappings until the match identity changes. Cache empty results briefly, not permanently, because live coverage can appear later.
26. Invalidate mapping on draw replacement, fixture participant change, materially changed schedule date, or provider mismatch.
27. Instrument unmapped and multiply-matched fixtures as explicit metrics/alerts.

Suggested interface

```
interface EventMapping {
  coreFixtureId: string;
  socketEventId: string | null;
  status: 'mapped'|'not_found'|'ambiguous'|'stale';
  mappedAt: Date;
  matchKey: string; // normalized players + date
}

async function resolveLiveEvent(fixture): Promise<EventMapping>
```

Tests

- Same players on different dates: correct event selected.
- Player order reversed between systems: mapping still succeeds if provider semantics allow.
- Replacement changes player: old mapping invalidated.
- event/get returns empty: status = not_found, no crash.
- Two candidates: status = ambiguous and fall back rather than guessing.

Discovery 6: There is no tournament-level Socket.IO room

Impact: Medium-high traffic/control **Priority:** P1

Provider-confirmed
MatchStat provides join-live-events-all or individual join-event subscriptions, but no join-tournament room.

Recommended MatchRead strategy

Subscribe to individual join-event rooms for active MatchRead matches. This keeps traffic scoped to the tournament/matches the product is displaying and gives clearer ownership of subscription lifecycle.

Socket subscription flow

```
active tournament fixtures
   |
   v
EventMapper for each live/near-live fixture
   |
   v
Mapped socket event IDs
   |
```

```
   +--> join-event(eventId)
   +--> join-event(eventId)
   +--> join-event(eventId)

On terminal match state / tournament navigation:
   -> unsubscribe/leave if supported by provider client
   -> stop local consumers
```

Implementation notes

- Define "active" narrowly: in progress plus a short pre-start window, rather than every tournament fixture for two weeks.
- Deduplicate event IDs so multiple MatchRead users/pages do not trigger duplicate backend subscriptions.
- Prefer one backend live-ingestion connection that fans out normalized updates internally, rather than opening a provider socket per browser user.
- If a global join-live-events-all feed is ever used, filter server-side by tournament and event ID immediately and measure throughput before relying on it.

Discovery 7: Socket replay does not exist

Impact: Critical reliability **Priority:** P0

Provider-confirmed
Missed websocket messages cannot be replayed after a disconnect. REST must be used to re-establish authoritative current state.

Required recovery state machine

```
CONNECTED
   | disconnect/error
   v
STALE
   | reconnect socket transport
   v
RESYNCING
   | GET /event/live-score for every active mapped event
   | overwrite current live cache/state
   v
SUBSCRIBED
   | re-join join-event rooms
   v
CONNECTED
```

Step-by-step coding work

28. Mark affected live state stale immediately on disconnect; do not continue presenting the last score as definitely current.
29. Reconnect Socket.IO with bounded backoff according to provider constraints.
30. Before trusting new incremental socket messages, fetch /event/live-score for each active event.

31. Overwrite the live cache with REST state instead of attempting to replay or patch from the last known socket sequence.
32. Re-join individual event rooms after connection restoration.
33. Only then mark the ingestion state healthy/current.
34. Expose lastRestSyncAt, lastSocketMessageAt, connectionStatus, and reconnectCount in monitoring.

Required chaos test

During a live rehearsal, intentionally kill the socket connection while score changes occur. Restore the connection and verify that MatchRead reaches the correct current score through REST even though the missing socket messages are never replayed.

Discovery 8: Live coverage is not guaranteed

Impact: Business/UX resilience **Priority:** P1

Provider-confirmed
MatchStat says not to assume 100% Extend Live coverage, particularly outside top-tier events. event/get may return empty.

Correction required

Treat "no live event" as a supported data state, not an exception. MatchRead must remain functional with bracket, picks, match status, and REST-driven score refresh where available.

Fallback hierarchy

```
1. Mapped Socket.IO event available
      -> live socket ingestion
      -> periodic REST sanity sync

2. event/get empty / no live mapping
      -> REST polling for available score/status endpoint(s)
      -> lower-frequency updates
      -> UI can indicate "updates may be delayed" if needed

3. No live score source available
      -> keep schedule/bracket/result state
      -> avoid fake live status
```

Step-by-step coding work

35. Represent liveCapability as a state such as socket | rest_poll | none rather than a boolean error flag.
36. When EventMapper returns not_found, schedule REST polling instead of throwing.
37. Use adaptive polling: faster while a match is in progress, slower before start, stop after terminal result.
38. Ensure settlement/result logic does not depend exclusively on socket events. Final results must come from an authoritative persisted result path.

39. Measure provider coverage by tour/event level so MatchRead can make evidence-based tournament support decisions.

Suggested metrics

- live_mapping_success_rate by tournament
- socket_covered_matches / active_matches
- rest_fallback_matches
- uncovered_matches
- average live update lag

Discovery 9: Grand Slam draw publication timing is not guaranteed

Impact: Critical launch automation **Priority:** P0

Provider-confirmed
MatchStat will not guarantee a fixed delay between official draw publication and API availability. MatchRead must detect availability/changes instead of scheduling one assumed fetch time.

Required draw polling worker

```
tournament approaching draw window
   |
   v
GET /draws on schedule
   |
   +-- empty/not usable -> keep polling
   |
   +-- first usable draw -> normalize + validate + publish
   |
   +-- later payload hash/version changed -> diff + apply replacements
   v
bracket availability state updated
```

Step-by-step coding work

40. Create draw ingestion as a scheduled/idempotent worker independent of user page requests.
41. Start polling during a configurable pre-event draw window; do not encode a single provider publication timestamp.
42. Hash or version the normalized draw payload so unchanged responses become cheap no-ops.
43. On first valid draw, run structural validation before publishing the tournament bracket.
44. Continue polling after first publication through the replacement-sensitive pre-match window.
45. When a draw change is detected, run the replacement diff logic from Discovery 4.
46. Keep bracket lock time as a MatchRead product rule based on tournament/match clocks - do not conflate "draw available" with "bracket locked."

Operational statuses

```
draw_status =
  'awaiting_provider' |
  'received_validating' |
  'published' |
  'changed' |
  'provider_error'
```

Cross-Cutting Implementation Architecture

Recommended backend modules

Module	Responsibility	Key provider interface
DrawProvider	Fetch raw official draw payload	GET /draws
DrawNormalizer	Convert provider draw records into MatchRead bracket nodes	provider payload -> internal types
DrawRefreshWorker	Poll, hash, diff, publish, detect replacements	GET /draws
FixtureEnricher	Attach fixture metadata without defining topology	Core fixture endpoints
EventMapper	Resolve core fixture -> Extend Live socket event ID	event/get
LiveSubscriptionManager	Join/manage active event rooms	join-event
LiveStateReconciler	Recover after disconnect using authoritative REST state	GET /event/live-score
LiveCoverageFallback	REST polling for unmapped/no-socket matches	REST score/status endpoints
ProviderObservability	Metrics, health, alerting, audit trail	all provider surfaces

Canonical ownership rules

- Bracket topology owner: Draw API.
- Bracket participant seed/entry designation owner: Draw API.
- Core match metadata owner: Core API.
- Real-time incremental score owner: Extend Live Socket.IO while connected.
- Authoritative recovery state after disconnect: REST live-score endpoint.
- MatchRead database owns normalized snapshots, mapping cache, user picks, lock rules, scoring, and audit history.

Do not couple these concerns

- Do not let websocket availability determine whether a bracket exists.
- Do not let a fixture ID double as a socket event ID.
- Do not let player identity define bracket-slot identity.

- Do not let draw publication time define bracket lock time.
- Do not let a socket event be the only path to final settlement.

Recommended Engineering Sequence

Goal
Fix the bracket trust boundary first, then live mapping/recovery, then graceful fallback and observability. This order reduces the chance of debugging live data on top of an incorrectly constructed bracket.

Phase 1 - Canonical draw ingestion (P0)

- Implement /draws client and normalization boundary.
- Prove draw-position semantics on known tournaments.
- Render bracket solely from normalized draw data.
- Store raw + normalized snapshots.

Phase 2 - Replacement-safe refresh (P0)

- Build scheduled draw polling.
- Hash/diff snapshots by stable slot.
- Detect player changes, seed/entry changes, and invalidate dependent mappings.

Phase 3 - Core-to-live EventMapper (P0)

- Implement event/get lookup using player1 + player2 + date.
- Validate candidates, persist mapping, support not_found/ambiguous states.
- Invalidate on participant/date changes.

Phase 4 - Live subscription manager (P0/P1)

- Join individual join-event rooms for active mapped matches.
- Normalize live messages into MatchRead state.
- Avoid provider socket-per-browser architecture.

Phase 5 - Disconnect recovery (P0)

- Mark state stale on socket loss.
- Reconnect, fetch /event/live-score, overwrite cache, rejoin rooms.
- Run deliberate disconnect chaos test.

Phase 6 - Coverage fallback (P1)

- When event/get is empty, switch to REST polling.
- Represent live capability state explicitly.
- Ensure result settlement remains independent of Socket.IO.

Phase 7 - Production observability (P0 before launch)

- Add metrics for draw freshness, mapping success, socket health, REST fallback, replacements, and stale state.
- Create founder/admin warning surface for provider anomalies.

Endpoint & Socket Cookbook

Important
The paths below are the endpoint/socket names surfaced in MatchStat's replies and our prior synthesis. The engineer should bind exact request parameters and response schemas to MatchStat's current documentation/account examples before production.

1. Fetch official bracket

```
GET /draws
```

Use as the canonical bracket source. Poll around draw publication and during the replacement window.

2. Resolve Extend Live event

```
event/get
  inputs: player1 + player2 + date
  output: candidate live event / eventId
```

Use inside EventMapper. Validate candidate identity; do not blindly accept ambiguous matches.

3. Subscribe to one live match

```
Socket.IO emit/join: join-event(eventId)
```

Preferred MatchRead pattern for active matches. Deduplicate subscriptions in the backend ingestion process.

4. Subscribe to all live tennis

```
Socket.IO: join-live-events-all
```

Available but not preferred for normal tournament-specific MatchRead ingestion. If used, filter immediately.

5. Re-sync live state after disconnect

```
GET /event/live-score
  input: mapped live event identity
  output: current authoritative live score/state
```

Fetch after socket reconnect before treating incremental socket updates as trustworthy again.

Reference Pseudocode

Draw refresh worker

```
async function refreshDraw(tournament) {
  const raw = await matchStat.get('/draws', tournament.providerArgs);
  const normalized = normalizeDraw(raw);

  if (!isUsableMainDraw(normalized)) {
    return markAwaitingProvider(tournament.id);
  }
```

```
  validateDraw(normalized);
  const previous = await loadLatestDrawSnapshot(tournament.id);

  if (!previous) {
    await publishInitialDraw(normalized);
    return;
  }

  const changes = diffByStableSlot(previous, normalized);
  if (changes.length === 0) return;

  await applyDrawChanges(changes);
  await invalidateAffectedLiveMappings(changes);
  await saveDrawSnapshot(normalized);
}
```

EventMapper

```
async function resolveLiveEvent(fixture) {
  const cached = await getCachedMapping(fixture.id);
  if (cached && mappingStillValid(cached, fixture)) return cached;

  const candidates = await matchStat.eventGet({
    player1: fixture.player1,
    player2: fixture.player2,
    date: fixture.startDate
  });

  const match = chooseValidatedCandidate(candidates, fixture);
  if (!match) return saveNotFound(fixture);
  if (match.ambiguous) return saveAmbiguous(fixture);

  return saveMapped(fixture.id, match.eventId);
}
```

Socket reconnect reconciler

```
socket.on('disconnect', () => markLiveStateStale());

socket.on('connect', async () => {
  setIngestionState('resyncing');
  const active = await listActiveMappedEvents();

  for (const event of active) {
    const restState = await matchStat.get('/event/live-score', { eventId:
event.socketEventId });
    await overwriteLiveState(event, restState);
    socket.emit('join-event', event.socketEventId);
  }

  setIngestionState('healthy');
});
```

Launch Test Matrix

Scenario	Expected behavior	Failure if…	Priority
Fixture list shuffled	Bracket unchanged	Topology depends on	P0

		fixture ordering	
Draw first appears	Validated bracket publishes once	Manual intervention required	P0
Player replacement	Same slot updates; mapping invalidated	Duplicate slot or stale player remains	P0
event/get empty	REST fallback activated	Match page errors	P1
event/get ambiguous	No guess; explicit ambiguous state	Wrong live feed attached	P0
Socket disconnect	State marked stale	UI silently freezes	P0
Reconnect after missed points	REST live-score restores current score	Missed socket events create wrong score	P0
No socket coverage	Bracket/picks/results continue	Tournament becomes unusable	P1
Draw unchanged on poll	No-op	Repeated writes/cache churn	P1
Final result arrives without socket	Settlement still succeeds	Scoring depends on socket	P0

Monitoring & Founder Dashboard Signals

- draw_last_success_at and draw_age_seconds
- draw_status and draw_change_count
- draw_validation_errors
- replacement_events_detected
- live_mapping_success_rate
- live_mapping_not_found_count
- live_mapping_ambiguous_count
- socket_connection_status and reconnect_count
- last_socket_message_at
- last_rest_resync_at
- rest_fallback_active_match_count
- live_state_stale_match_count

Launch alert principle
A provider problem should create a visible operational state before it becomes a user report. The minimum warning conditions are: draw stale/unavailable near publication window, replacement detected, mapping ambiguity, socket disconnected, and live state stale beyond the allowed recovery interval.

Engineer Handoff Checklist

- [ ] Bracket topology is sourced only from /draws.
- [ ] Raw provider draw values are preserved alongside normalized slots.
- [ ] Draw-position semantics are validated against real tournament samples before global formulas are frozen.
- [ ] seed1/seed2 are stored as official seeds; WC/PR/Q/LL are separate entry types.
- [ ] Draw refresh worker diffs participant identity by stable slot and detects replacements.
- [ ] Replacement invalidates affected fixture/live mapping.
- [ ] EventMapper resolves Core fixture -> Socket eventId through event/get using players + date.
- [ ] Mapper handles mapped, not_found, ambiguous, and stale states.
- [ ] Backend subscribes to individual join-event rooms for active mapped matches.
- [ ] Socket loss marks live state stale.
- [ ] Reconnect triggers /event/live-score overwrite before normal live state is considered healthy.
- [ ] No replay assumption exists anywhere in recovery logic.
- [ ] event/get empty activates REST fallback instead of an error path.
- [ ] Final settlement can succeed without receiving a socket event.
- [ ] Draw polling detects first publication and later changes without assuming an exact Grand Slam API delay.
- [ ] Key provider health metrics are visible to engineering/admin before launch.

Final Architecture Rule

The trust boundary
Draw API tells MatchRead where players belong. Core API tells MatchRead what the scheduled match is. EventMapper tells MatchRead which Extend Live event corresponds to that match. Socket.IO supplies incremental live updates. REST repairs truth after gaps. MatchRead owns normalization, persistence, user picks, locks, scoring, and resilience.

Provider confirmation vs. MatchRead recommendation

Provider-confirmed items in this guide are based on the MatchStat replies supplied by the founder: use /draws for bracket structure; a draw position field exists; seed1/seed2 are official seeds; lucky-loser/replacement handling is observed as changed player data rather than a dedicated event; core and socket IDs differ; the event/get player/date flow is used for live resolution; there is no tournament socket room; replay is unavailable; coverage is not guaranteed; and draw-publication delay is not guaranteed. Everything concerning module names, schemas, caching policy, polling cadence, state machines, UI wording, metrics, and exact internal data models is a MatchRead engineering recommendation and should be adapted to the existing codebase.